LegalEase

2. Requirement Analysis Phase

AI-Powered Legal Document Generator

Objective

Translate the project idea into functional, technical, and user-interface requirements.

Functional Requirements

- Accept document type, parties, terms and effective date.
- Generate legal content through the Gemini integration.
- Expose a POST /generate backend endpoint.
- Display generated text in the frontend.
- Allow users to edit generated content.
- Download documents as .txt, .docx and .pdf.
- Support formatting such as titles, headings, terms tables, logos and footers.

User Journey

- User opens the LegalEase Streamlit interface.
- User enters document type, parties, terms and effective date.
- User selects Generate Document.
- Frontend sends the information to the FastAPI backend.
- Backend validates the request and calls GeminiDocumentGenerator.
- Generated content is returned and shown in a preview.
- User edits the document if required.
- User downloads the final document in the required format.

Data Flow

- Streamlit Frontend → FastAPI /generate → Pydantic validation → GeminiDocumentGenerator → Gemini API → Generated legal text → Streamlit preview/editing → TXT/DOCX/PDF export.

Technology Requirements

- Python 3.10+.
- FastAPI and Uvicorn for backend services.

- Streamlit for frontend UI.
- Google Generative AI SDK for Gemini integration.
- python-docx for Word document generation.
- FPDF for PDF generation.
- Pillow for image/logo handling.
- requests and python-dotenv for supporting functionality and environment configuration.